

Étapes réalisées — piste non supervisée

Compte rendu pas à pas de la journée du 7 août 2026 : ce qui a été construit, dans quel ordre, et ce que les mesures ont révélé en cours de route.

Point de départ

Le pipeline WALLIX → Wazuh → extraction fonctionnait, et un générateur de sessions bénignes synthétiques existait. Mais aucun modèle n'avait encore été entraîné, et la couche de métadonnées (`identity_pool.json`) était construite sans être consommée par quoi que ce soit : chaque session générée portait un seul utilisateur et une IP fictive par persona.

Étape 1 — Brancher la couche de métadonnées

Réécriture de `generate_benign.py` pour draper chaque session synthétique sur une identité réelle du pool. Chaque utilisateur possède, figé une fois pour toutes : 1 à 2 IP de poste dans le sous-réseau de son équipe, 2 à 4 hôtes habituels tirés selon l'affinité de son persona, et sa propre distribution d'heures de travail.

	Avant	Après
Utilisateurs	1 (test-ssh)	16
IP clientes distinctes	4 (placeholder)	107
Cibles	1	15
Sessions hors-heures	0 %	14,6 %

Le bénin porte **volontairement** du bruit sur chaque axe : 8 % de sessions VPN, 2 % depuis un serveur de rebond, quelques week-ends, quelques emprunts légitimes du compte privilégié. Sans ce recouvrement, « IP inhabituelle » ou « hors-heures » sépareraient à eux seuls le bénin de l'attaque, et le modèle apprendrait les métadonnées au lieu du comportement.

Étape 2 — Construire la piste non supervisée

Deux scripts, plus une étape méthodologique intercalée.

2.1 Le jeu d'évaluation isolé

`build_eval_set.py` joint la vérité terrain aux sessions extraites via le marqueur `echo TAG:` — le seul lien possible, puisque WALLIX attribue un identifiant opaque sans rapport avec le scénario joué. Les sessions sans marqueur ou en erreur sont **comptées et rapportées**, jamais écartées en silence.

2.2 La porte de domaine

On entraîne sur du généré et on évalue sur du réel. Cela ne fonctionne que pour les features dont la distribution **bénigne** est la même des deux côtés. Chaque feature candidate passe donc un test de Kolmogorov-Smirnov entre bénin généré et bénin réel. Une feature qui échoue est un **marqueur de domaine** : le détecteur signalerait une session parce qu'elle est réelle, et le `precision@k` mesurerait la couture entre les deux jeux de données au lieu d'une anomalie.

2.3 Le bake-off

`train.py` entraîne ECOD, COPOD, Isolation Forest, HBOS et PCA sur le bénin généré uniquement, plus un baseline MAD **obligatoire** (z-score robuste). Configuration A : features comportementales seules, sans les drapeaux de mots-clés — ces drapeaux sont une réimplémentation des règles Wazuh, et les fournir au modèle reviendrait à lui faire réapprendre les règles.

Étape 3 — Le premier bake-off révèle deux artefacts

Le premier résultat semblait excellent. Il ne l'était pas : deux défauts de collecte faisaient que le détecteur lisait des artefacts de script plutôt que du comportement. Aucun des deux n'a été trouvé par relecture du code — les deux ont été trouvés **par la mesure**.

Artefact n°1 — `unique_command_ratio` valait 1,00 partout

Une session était une liste fixe de commandes distinctes jouée une seule fois ; un humain se répète. La porte de domaine a rejeté `unique_command_ratio` (KS 0,94) et `command_entropy` (KS 0,69) : deux features sur cinq perdues.

Artefact n°2 — une attaque était à 100 % des commandes d'attaque

`avg_command_length` atteignait à elle seule une AUC de 0,832. Le détecteur lisait « commande longue », pas du comportement. La tactique *recon* (14,3 caractères de moyenne) était indistinguishable du bénin et passait totalement inaperçue.

Décision : corriger le collecteur, jamais calibrer le générateur sur un artefact de script. Faire coïncider le générateur avec un `unique_command_ratio` de 1,00 aurait appris au modèle que « bénin = toutes les commandes distinctes », et un utilisateur qui répète une commande serait devenu anormal pour la mauvaise raison.

Étape 4 — Corriger le collecteur

Les sessions sont désormais **composées** au lieu d'être rejouées.

Composition	Contenu
<code>sampled</code> (bénin)	longueur variable (4–14) et répétition des commandes
<code>full</code>	le scénario seul — le cas bruyant, conservé pour le gradient d'intensité
<code>diluted</code>	le scénario entier, avec du bénin intercalé entre ses commandes ; l'ordre relatif est préservé, donc les paires créer-puis-supprimer nettoient toujours
<code>minimal</code>	1 à 2 commandes d'attaque en lecture seule enfouies dans une session bénigne

Sûreté : le mode *minimal* ne pioche que dans une liste en lecture seule par construction — aucune commande n'y crée d'état, donc une session interrompue avant son nettoyage ne peut rien laisser derrière elle. Les scénarios qui créent de l'état ne sont jamais fragmentés.

Effet mesuré sur télémétrie réelle :

	Avant	Après
Écart bénin/attaque en longueur de commande	25,5 car.	1,9 car.
<code>unique_command_ratio</code> (bénin réel)	1,00	0,77
Features admises par la porte de domaine	3 / 5	5 / 5

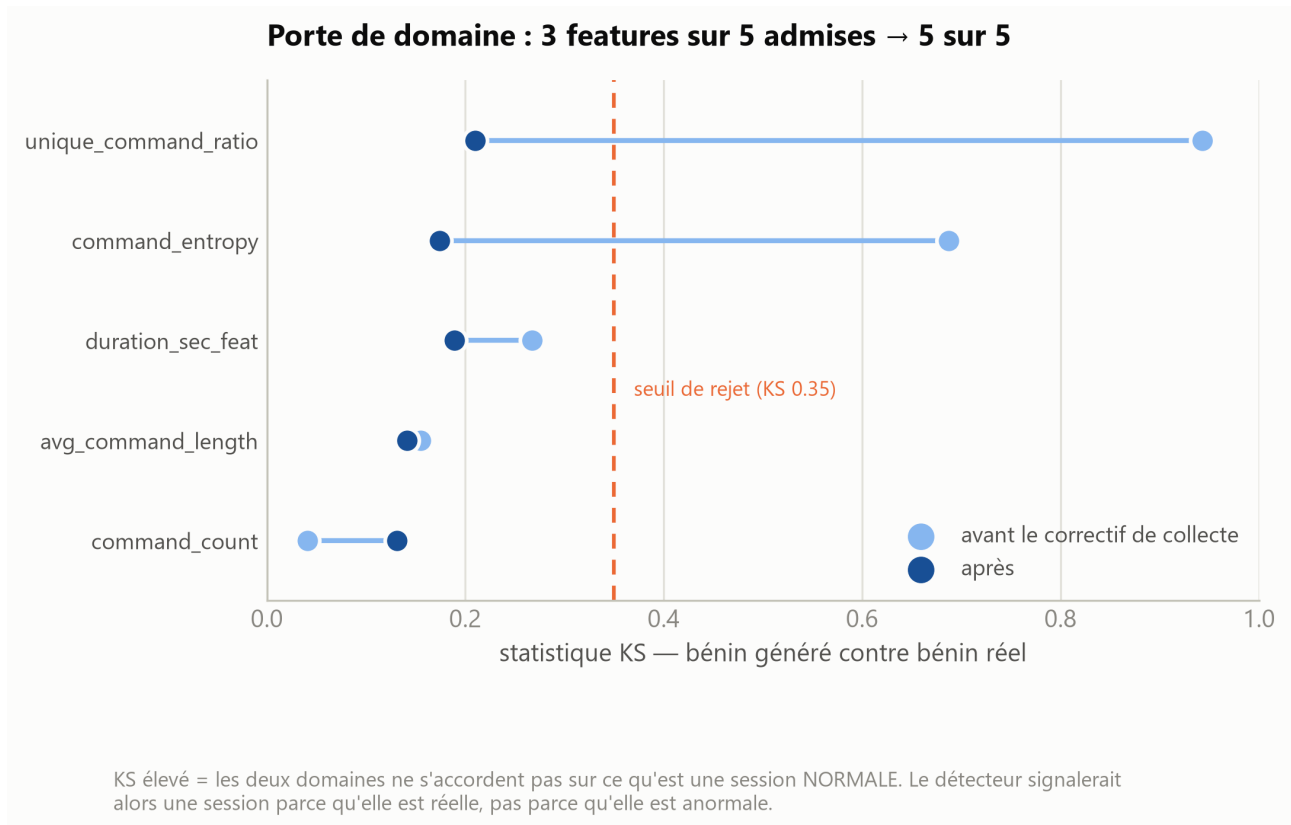


Figure 1 — Statistique KS par feature, avant et après le correctif. Au-delà du seuil, la feature décrit deux populations différentes et ne peut pas porter un verdict inter-domaines.

Étape 5 — Un incident de sûreté, et sa correction

Une session collectée a exécuté « `mv /usr/local/bin/* /opt/bin/` » en root

Aucun dégât — uniquement parce que `/usr/local/bin` était vide. Rien ne l'avait empêché. Deux causes cumulées : le corpus de commandes bénignes avait été filtré contre les règles de détection mais **jamais contre la destructivité**, et le chemin d'accès à ce corpus était périmé depuis une restructuration du dépôt — il n'avait donc jamais été réellement chargé. Le défaut s'est révélé au moment où le chemin a été corrigé.

Un module `command_safety.py` devient la **définition unique**, importée par les deux côtés pour deux raisons différentes qui exigent la même règle :

- **Le collecteur — sûreté.** Le remplissage bénin s'exécute vraiment, en root, sur un hôte partagé.
- **Le générateur — parité.** Rien n'y est exécuté, donc rien n'y est dangereux. Mais si le générateur peut produire une commande que le collecteur ne peut plus produire, les deux vocabulaires divergent — et cette divergence réapparaît comme un marqueur de domaine.

Deux classes sont bloquées : la **mutation d'état** et la **non-termination** (`free -s 1, tail -f, ping sans -c`). Cette seconde classe est la plus sournoise : elle bloquerait la session jusqu'au délai d'expiration et **tronquerait silencieusement** la collecte au lieu de planter. 1669 commandes rejetées sur environ 3200.

Étape 6 — Re-collecte et recalibration

172 sessions réelles collectées en deux lots, sans échec. Puis une recalibration complète du côté généré : le bénin réel ayant changé, la calibration du générateur était devenue périmée — et cet écart serait réapparu comme un marqueur de domaine.

Jeu d'évaluation	Avant	Après
Sessions réelles labellisées	79	256
Attaques	45	49
Taux de base (part d'attaques)	57,0 %	19,1 %

Un piège évité de justesse

La porte de domaine compare **bénin contre bénin uniquement** : elle est structurellement aveugle à une contamination de la classe **attaque**. Après avoir corrigé le côté bénin, 45 des 58 attaques du jeu d'évaluation dataient encore d'avant le correctif — les chiffres portaient donc sur une classe attaque artificielle à 78 %. D'où l'option - -composed-only, qui ne conserve que les sessions postérieures au correctif.

Étape 7 — Résultat sur base propre

Modèle	ROC-AUC	P@25	P@50	Gain @50
PCA	0,778	0,72	0,60	×3,13
ECOD	0,755	0,56	0,54	×2,82
Isolation Forest	0,775	0,60	0,52	×2,72
HBOS	0,749	0,44	0,48	×2,51
MAD (baseline)	0,798	0,68	0,48	×2,51
COPOD	0,719	0,36	0,42	×2,19

Un classifieur aléatoire obtient 0,191 à tout k . À $k = 50$: 30 attaques trouvées sur 49, pour 20 faux positifs sur 207 sessions bénignes.

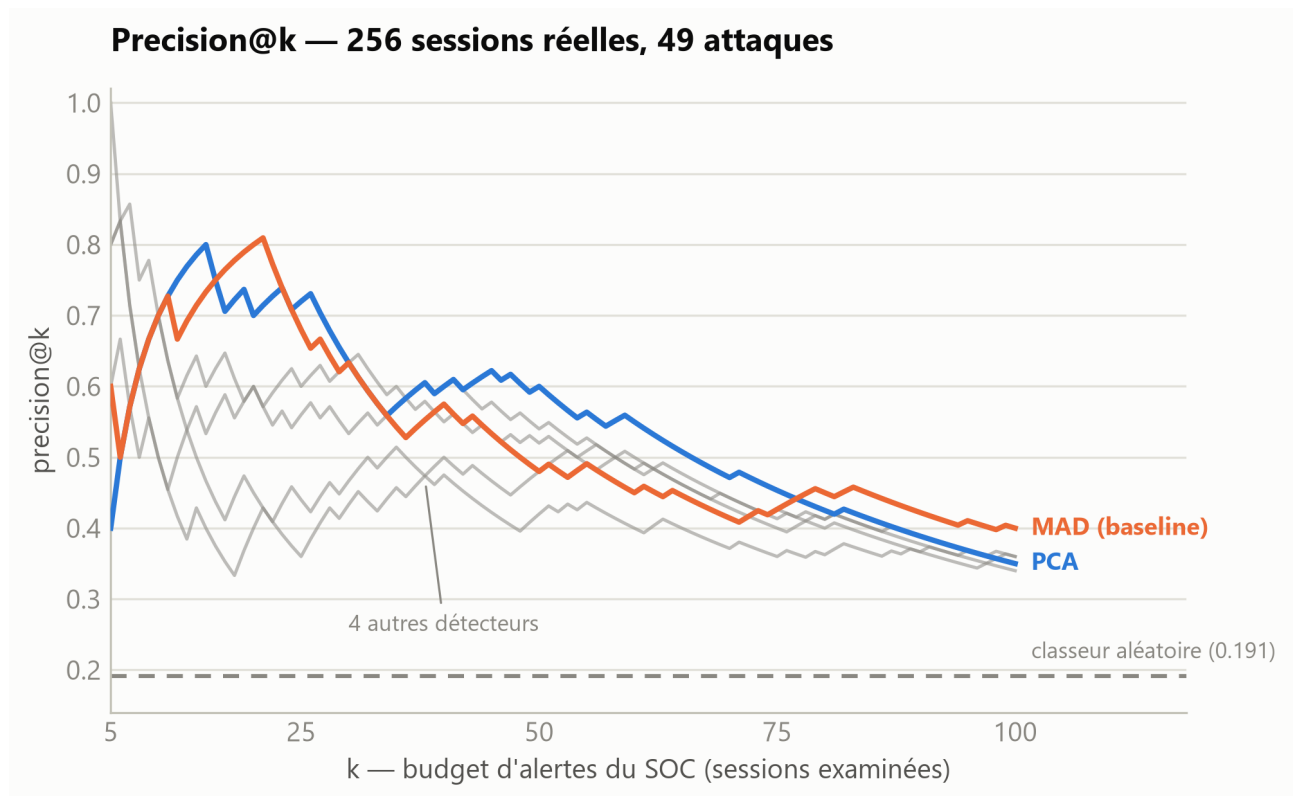


Figure 2 — Precision@k. Les deux courbes se croisent : le baseline est meilleur à petit k, PCA à k moyen.

Deux points à défendre

1. Le baseline MAD gagne le ROC-AUC et perd le precision@k. Les deux métriques se contredisent sur les mêmes données — c'est une justification *empirique* du choix de precision@k, pas un argument emprunté à la littérature. Choisir le ROC-AUC aurait fait déployer le mauvais modèle.
2. Avant le correctif de collecte, HBOS menait à 0,862. Cet avantage était l'artefact, pas de l'apprentissage. Le chiffre a baissé *parce que l'évaluation est devenue honnête*.

Étape 8 — Le résultat structurant

Décomposer le rappel par furtivité d'attaque est ce qui transforme « le modèle marche à 61 % » en une information exploitable.

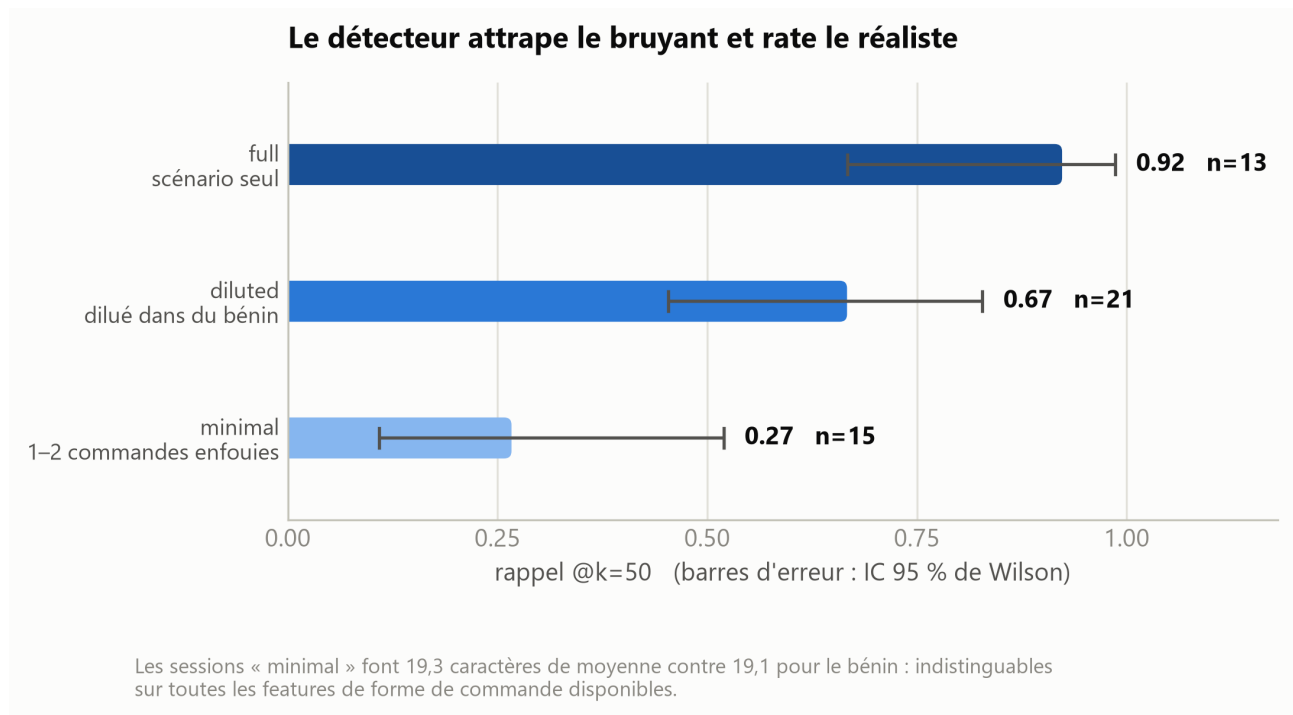


Figure 3 — Rappel par composition d'attaque, avec intervalles de confiance de Wilson à 95 %.

Les sessions *minimal* font 19,3 caractères de moyenne contre 19,1 pour le bénin : sur **toutes** les features de forme de commande disponibles, elles sont indistinguables. Le détecteur attrape le cas bruyant et rate le cas réaliste — or c'est le cas réaliste qui compte, puisqu'un véritable acteur interne ne lance pas un scénario d'attaque complet, il glisse deux commandes dans sa journée de travail.

Ce n'est pas un échec de réglage mais une **limite mesurée**, et elle motive directement les features contextuelles (heure, IP source, cibles distinctes sur 24 h) — que le laboratoire ne peut pas fournir côté réel avec une seule identité, une seule IP source et une seule cible.

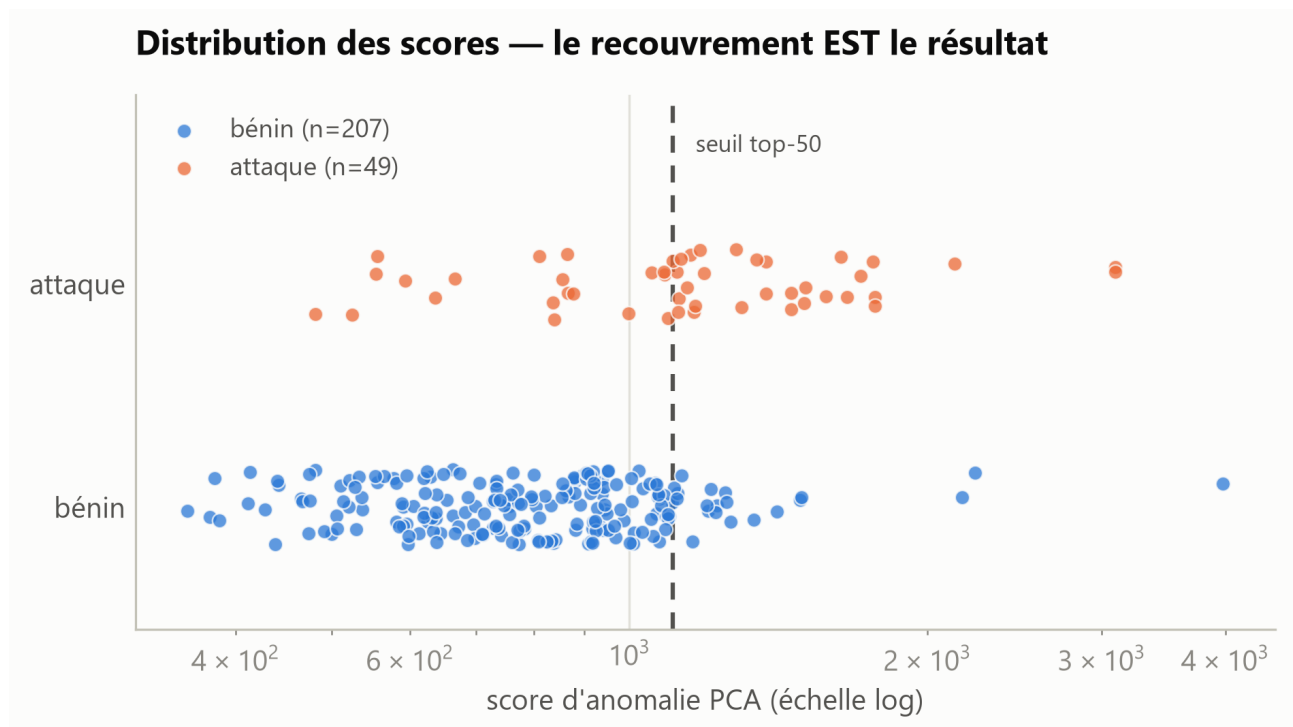


Figure 4 — Chaque session, par classe. Le recouvrement est le résultat : le modèle ne sépare pas proprement, il biaise le classement.

Étape 9 — Figures et documentation

Cinq figures dans `visualisation/`, en PNG et PDF. Le script **relit les artefacts produits par l'entraînement** au lieu de recalculer : les figures ne peuvent pas diverger des chiffres du rapport. Le journal de décisions et le guide du projet ont été mis à jour en parallèle.

Ce qui reste à faire

Priorité	Chantier	Ce que cela débloque
1	<code>rule_baseline.py</code> — Pool A/B, matrice 2×2, test de McNemar	La thèse elle-même. Sans lui, il y a un modèle ML sans point de comparaison, et « les règles ne suffisent pas » n'est pas démontré.
2	Davantage de sessions <i>minimal</i>	Resserre l'intervalle de confiance du 0,27 : temps machine, pas temps de développement.
3	Chronologies par utilisateur dans le générateur	Toutes les features à fenêtre 24 h, aujourd'hui vides de sens.
4	Features contextuelles	Le seul levier crédible sur le cas <i>minimal</i> .

Ce qu'il ne faut pas faire : ajouter d'autres détecteurs — le meilleur ne bat qu'à peine un z-score, donc le levier est dans les features, pas dans les algorithmes ; et régler des seuils pour améliorer les chiffres, « le ML réglé a battu les règles non réglées » étant la critique la plus facile à formuler en soutenance.